

And_Cut — Video Loading Analysis & Improvement Plan

The Honest Truth First

The client's goal — **every video plays instantly, seamlessly, no loading, no buffering** — is not fully achievable on the web. Here's why in one sentence:

Video data is large (MBs), network pipes are narrow (KB/s on slow connections), and browsers deliberately throttle cross-origin media to protect bandwidth.

That said, the site already does a **lot of smart things**, and there are still concrete improvements you can make. This document covers both.

What the Site Already Does Well (Don't Remove These)

Mechanism	Where	What it achieves
Vimeo background-mode iframe	Hero (desktop)	Vimeo CDN + adaptive bitrate = fastest possible external streaming
Local <code><video></code> with <code>preload="auto"</code>	Hero (mobile)	Same-origin video bypasses cross-origin iframe suspend on iOS
Preloader holds until assets ready	<code>Preloader.tsx</code>	Hides buffering behind a polished animation
Showreel pre-buffered via <code><video></code> probe	<code>Preloader.tsx</code> DESKTOP	Video is already playing when preloader fades
750 KB partial blob pre-fetch	<code>videoCache.ts</code> + <code>LocalBlobVideoCard.tsx</code>	UGC/DVC cards play from RAM — zero network on first play

VimeoFacadeCard thumbnail → iframe transition	VimeoFacadeCard.tsx	User NEVER sees a black box — always a thumbnail
IntersectionObserver 500px rootMargin	Carousel/index.tsx	Vimeo iframes start buffering 500px before user reaches them
Cache-Control: immutable on all video folders	next.config.mjs	Repeat visitors get all assets from browser cache instantly
Preload:metadata	Beyond_vertical.tsx	Large horizontal videos only download enough to show first frame

Section-by-Section Breakdown & Improvements

1. Hero Section (Desktop) — Vimeo iframe

Current behaviour: Vimeo iframe is injected at page load. It buffers during the preloader animation. By the time preloader ends, it should be playing.

Why it still stutters:

- Vimeo is a **cross-origin** server. You cannot control when it starts delivering frames.
- On slow 4G / bad Wi-Fi the iframe may not have buffered even after the full preloader duration (~4–6 s).
- Vimeo's `quality=auto` is good but can still serve a low-quality first chunk.

Improvements you can make:

- ☒ **Force low quality first** — Change `quality=auto` → `quality=360p` in `vimeoBgSrc()`. 360p starts playing in ~30% of the time 1080p does. It still looks great as a full-bleed background. ([Hero/index.tsx](#))
- ☒ **Add a poster image as fallback** — Keep a static screenshot of the first frame of the hero video as a WebP. Show it behind the Vimeo iframe as a CSS `background-image`. If

Vimeo hasn't loaded yet, the user sees a crisp still image instead of black. This is the single biggest visual win.

-  **Extend preloader hard timeout on slow connections** — Add a Network Information API check. If `navigator.connection.effectiveType === "2g"` or `"slow-2g"`, extend `finalDelay` to 3–4 seconds extra.

2. Hero Section (Mobile) — Local MP4

Current behaviour: `/ANDCUT_VDS/MobileHero.mp4` at **~15 MB**. Plays via a local `<video>` with `preload="auto"`.

Why it still stutters:

- 15 MB is very large for mobile. On 4G (~5 Mbps), that's **24 seconds to download**. Even with streaming, playback starts around 2–3 MB, but the buffer will empty mid-video on slow connections.
- `preload="auto"` tells the browser to download the full file — iOS/Android may still throttle this for battery reasons.

Improvements you can make (most impactful):

-  **Re-encode MobileHero.mp4 at lower bitrate** — This is the #1 fix. The file should be **under 4 MB** for mobile. Use `ffmpeg`:

```
ffmpeg -i MobileHero.mp4 -c:v libx264 -crf 28 -preset slow -vf
scale=720:-2 -an -movflags +faststart MobileHero_mobile.webm
```

Target: 720p, no audio, CRF 28, WebM. Should land at 3–5 MB. That's a 3–4× smaller download.

-  **Add a poster frame** — `<video poster="/hero-mobile-poster.webp">`. The poster shows instantly while the video buffers. Smooth transition, zero black screen.
-  **Use `<source>` with WebM first, MP4 fallback** — Safari only, Chrome/Android prefers WebM which is 20–30% smaller.

3. Preloader Showreel (Desktop) — Cloudinary video

Current behaviour: Cloudinary video (~unknown size, fetched from CDN) is pre-loaded using a `<video>` probe. Plays inside the preloader as the middle card.

Why it can delay the preloader:

- If the Cloudinary video is large (>5 MB), `canplay` won't fire until the browser downloads enough to start — and on slow connections this can delay the preloader start by 3–5 seconds.
- `ASSET_TIMEOUT_MS = 8000ms` desktop means the preloader waits up to 8 seconds just for this video.

Improvements:

- ☒ **Create a low-quality poster frame** for the showreel and show it as fallback during loading. The preloader animation still plays beautifully — worst case the center card shows a still image.
 - ☒ **Reduce `ASSET_TIMEOUT_MS` to 4000ms** on desktop. If Cloudinary hasn't delivered enough in 4s on desktop, something is wrong with the connection — let the preloader go anyway. ([Preloader.tsx](#))
-

4. Carousel — UGC & DVC Cards (LocalBlobVideoCard)

Current behaviour: 750 KB partial blob pre-fetched per video during preloader. Cards play from RAM.

This is already the BEST solution possible for local files.

Minor improvements:

- ☒ **Increase partial blob to 1.2 MB** (from 750 KB) — 1.2 MB covers ~6–10 seconds of WebM at typical UGC bitrates. This reduces the frequency of the "ended → loop restart" flash. Change `PARTIAL_BYTES` in [videoCache.ts](#).
 - ☒ **Only pre-fetch the active tab's videos first** — Currently ALL UGC videos are fetched during preload. Instead, fetch only the first tab (UGC), then fetch others lazily after page loads. This reduces total preloader blocking.
-

5. Carousel — Micro Drama & some DVC (VimeoFacadeCard)

Current behaviour: Thumbnail shows immediately. Iframe injected when section is 500px away. Video fades in after Vimeo fires "ready".

Why it stutters:

- Vimeo iframes are sandboxed. The browser doesn't start loading them until they are injected into the DOM.
- On slow connections, "ready" fires 3–8 seconds after iframe injection.
- The 5-second fallback means the user may see thumbnail → fade → still-buffering-video.

Improvements:

- ☒ **Increase rootMargin to "1000px 0px"** — Start Vimeo buffering 1000px before section. Gives 2× the head-start. ([Carousel/index.tsx](#))
 - ☒ **Keep thumbnail visible until the video has played at least 1 frame** — Instead of fading on "ready", fade on the Vimeo "play" event, which means frames are actually being rendered. The current code already handles "play" — keep it.
 - ☒ **Migrate Micro Drama from Vimeo to local WebM blobs** — The `/micro_drama/` MP4 files are already in `/public`. If you convert them to WebM and make them `useLocalCard: true` in [services.ts](#), they play from local blob just like UGC. This completely eliminates Vimeo latency for this tab. **This is the biggest remaining improvement for the carousel.**
-

6. Beyond Vertical — Horizontal Videos

Current behaviour: Large `.webm` files (CNBC.webm = 15 MB, horizontal2.webm = 10 MB) with `preload="metadata"`. IntersectionObserver starts playing when 15% visible.

Why it stutters:

- `preload="metadata"` only downloads the first ~100 KB (enough to read video dimensions/duration). So when the user scrolls to the section, the browser starts a cold download of a 10–15 MB file.
- These are desktop-only (`hidden lg:flex`) which helps, but desktop connections aren't always fast.

Improvements:

- **✓ Use `preload="auto"` for the first card only** — The first horizontal video card is always visible. Let it pre-buffer. Keep others on `"metadata"`.
 - **✓ Re-encode at lower bitrate** — 15 MB for a background-preview video is excessive. Target 3–5 MB at 720p, still looks great at 16:9 card size.
 - **✓ Add a poster image** — `<video poster="/ad_films/CNBC-poster.webp">` — shows the first frame immediately. The video loads in background. No black flash.
-

7. AlternatingText — Phone Mockup Videos (ANDCUT_GIFs)

Current behaviour: `preload="none"` — videos only load when the browser decides. In a pricing section, these phone mockups need to play immediately when visible.

Improvement:

- **✓ Change to `preload="metadata"` or add `IntersectionObserver`** — Same pattern as `Beyond_vertical.tsx`. Load on visibility, not on page load. This section is commented out (`return <></>`), but when you re-enable it, apply this.
-

The Replacement Strategy: GIF/APNG Instead of Video (where applicable)

For **very short clips used as ambient/decorative loops** (e.g. the pricing section phone mockups in `ANDCUT_GIFs`), consider **animated WebP** instead of video:

- Animated WebP: loads like an image, plays automatically, no browser autoplay policy issues
- Works even behind preloader overlays (unlike cross-origin iframes)
- **Con:** No audio, limited to ~5–10s loops, larger file than a short WebM

Use `ffmpeg` to convert:

```
ffmpeg -i input.webm -vf "fps=12,scale=320:-2" -loop 0 output.webp
```

Priority List (Highest ROI first)

#	Change	Effort	Impact
1	Re-encode MobileHero.mp4 → WebM ≤4 MB	Low (ffmpeg)	🔥 Huge
2	Add poster images to all <code><video></code> tags	Low	🔥 Huge (no black screens ever)
3	Migrate Micro Drama to <code>useLocalCard: true</code>	Medium	🔥 High
4	Force Vimeo Hero to <code>quality=360p</code>	1 line	High
5	Increase Vimeo IntersectionObserver <code>rootMargin</code> to 1000px	1 line	Medium
6	Re-encode Beyond Vertical videos to ≤5 MB	Low	Medium
7	Increase blob partial size to 1.2 MB	1 line	Low-Medium
8	Reduce desktop ASSET_TIMEOUT_MS to 4000	1 line	Low

Cons to Explain to the Client

This section is meant to be shared with the client verbatim.

Why "instant, seamless, no buffering" is not fully achievable on the web:

- Physics of data transfer** — A 10 MB video at 5 Mbps (average Indian 4G) takes 16 seconds to download. We can stream it so playback starts at 1–2 seconds, but on a slow connection the buffer will empty.
- Browser autoplay policies** — All modern browsers (Chrome, Safari, Firefox) block autoplaying videos with audio. Muted autoplay works, but browsers can still throttle cross-origin video loads (like Vimeo iframes) when they are not the focused tab.
- Cross-origin video (Vimeo) cannot be cached locally** — We cannot pre-download Vimeo videos. They are served from Vimeo's servers and stream only when the browser is ready. This is a legal and technical limitation of embedding external video.

4. **Mobile browsers are especially aggressive** — iOS Safari suspends video loading for cross-origin iframes that are not in the active viewport. This is intentional power-saving behaviour we cannot override.
5. **The site already uses the most advanced techniques available** — Partial blob caching, facade thumbnails, IntersectionObserver pre-loading, preloader buffering, adaptive bitrate (Vimeo). There is no magic API that bypasses network physics.

What we CAN guarantee with the improvements above:

-  **No black boxes ever** — Thumbnails and poster frames are always shown
-  **Local videos (UGC, DVC) play from RAM** — Zero buffering on those cards
-  **Repeat visitors get everything from browser cache** — Instant on second visit
-  **Hero video starts within 2–3 seconds** on a typical 4G connection
-  **The experience feels seamless** — even if bits are still downloading underneath